

Experiment 5

Study of Addressing Modes by Analyzing the Execution of a C Program using GNU Debugger (GDB)

Experiment No.	5	Subject	COA
Branch	CSE	Name	Anshul Modi
Roll No.	241210021		

1. Aim / Objective

To study different **addressing modes** of instructions by analyzing the execution of a C program step-by-step using the **GNU Debugger (GDB)**. All five addressing modes — Immediate, Direct, Data Segment (Global), Indirect, and Indexed — are demonstrated in a single unified C program.

2. Theory

An **addressing mode** specifies how the operand of an instruction is determined. Modern processors support multiple addressing modes to provide flexibility in accessing data from registers, immediate constants, memory locations, and pointers. Understanding addressing modes is fundamental to computer architecture as they directly influence instruction encoding, program efficiency, and memory access patterns.

Addressing Mode	Code Line	Description
Immediate	result = a + 5	Constant 5 is embedded directly in the instruction
Direct	result = a + b	Operands accessed directly from named memory locations
Data Segment	result = x	Global variable x accessed from the data segment (RIP-relative in x86-64)
Indirect	ptr = &a; result = *ptr	Address of 'a' stored in ptr; value obtained by dereferencing
Indexed	result = arr[1]	Effective address = base address of arr + index × element size

GDB allows us to step through a C program line by line, print variable values, inspect memory addresses, view CPU registers, and disassemble machine code — making it an ideal tool to observe how each addressing mode operates at the hardware level.

3. Source Code — lab5_addressing_modes.c

A single C program demonstrating all five addressing modes:

```

5  #include <stdio.h>
6
7  int main(){
8      int a=10, b=20;
9      int result;
10     int arr[3]={1,2,3};
11     int *ptr;
12
13     result = a + 5;      // Immediate addressing
14     result = a + b;     // Direct addressing
15     ptr = &a;           // Indirect addressing (load address)
16     result = *ptr;      // Indirect addressing (dereference)
17     result = arr[1];    // Indexed addressing
18     printf("Result : %d", result);
19     return 0;
20 }
21

```

Fig. 1 – Source code of lab5_addressing_modes.c with all addressing modes annotated.

4. GDB Commands Used

GDB Command	Purpose
gcc -g lab5_addressing_modes.c -o out1	Compile with debug symbols
gdb ./out1	Launch GDB
break main	Set breakpoint at main
run	Start execution
next	Step to next source line
print a / print b / print x	Inspect variable values
print &a	Print memory address (Direct)
print ptr / print *ptr	Inspect pointer (Indirect)
print arr[1]	Inspect array element (Indexed)
info registers	View all CPU register values
disassemble main	View assembly for main function

5. Procedure

1. Write the combined C program (lab5_addressing_modes.c) covering all five addressing modes.
2. Compile with debug symbols: gcc -g lab5_addressing_modes.c -o out1
3. Launch GDB: gdb ./out1
4. Set a breakpoint at main: break main, then start with run.
5. Use 'next' to step through each line one at a time.
6. After 'result = a + 5', use 'print result' to confirm Immediate addressing (result = 15).
7. After 'result = a + b', use 'print result' to confirm Direct addressing (result = 30).

8. After 'result = x', use 'print result' to confirm Data Segment access (result = 10).
9. Use 'print &a;' to view the memory address, then 'print ptr' and 'print *ptr' to verify Indirect addressing.
10. Use 'print arr[1]' to confirm Indexed addressing (value = 2).
11. Use 'info registers' to view all CPU register values (rax, rsp, rip, eflags, etc.).
12. Use 'disassemble main' to view the x86-64 assembly and correlate each C line with its addressing mode.

6. GDB Session Screenshots

```
nitdelhi@w2:~/coalab$ vi lab1.c
nitdelhi@w2:~/coalab$ gcc -g lab1.c -o out1
nitdelhi@w2:~/coalab$ ./out
bash: ./out: No such file or directory
nitdelhi@w2:~/coalab$ ./out1
Result :2nitdelhi@w2:~/coalab$ gcc -g lab1.c -o out1
nitdelhi@w2:~/coalab$ gdb ./out1
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./out1...
(gdb) break main
Breakpoint 1 at 0x1169: file lab1.c, line 2.
(gdb) run
Starting program: /home/nitdelhi/coalab/out1

Breakpoint 1, main () at lab1.c:2
2      int main(){
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/coalab/out1

Breakpoint 1, main () at lab1.c:2
2      int main(){
(gdb) next
3          int a=10,b=20;
(gdb) next
5          int arr[3]={1,2,3};
(gdb) next
7          result=a+5;
(gdb) print a
$1 = 10
(gdb) print &a
$2 = (int *) 0x7ffffffdd54
```

Fig. 2 – GDB session: compilation, breakpoint at main, stepping through lines, printing variable values (a=10) and memory address of 'a' (0x7ffffffdd54).

```

(gdb) print ptr
$3 = (int *) 0x0
(gdb) print *ptr
Cannot access memory at address 0x0
(gdb) print arr[1]
$4 = 2
(gdb) next
8           result=a+b;
(gdb) next
9           ptr=&a;
(gdb)
10          result=*ptr;
(gdb) print ptr
$5 = (int *) 0x7fffffffdd54
(gdb) print *ptr
$6 = 10
(gdb) next
11          result=arr[1];
(gdb) next
12          printf("Result :%d",result);
(gdb) next
13          return 0;
(gdb) next
15      }
(gdb) info registers
rax          0x0                0
rbx          0x55555555210       93824992236048
rcx          0x0                0
rdx          0x0                0
rsi          0x55555555600e      93824992239630
rdi          0x7fffffff7b07e0    140737353811936
rbp          0x7fffffffdd80      0x7fffffffdd80
rsp          0x7fffffffdd50      0x7fffffffdd50
r8           0x0                0
r9           0x9                9
r10          0x55555555600e      93824992239630
r11          0x7fffffffdc27      140737488346151
r12          0x55555555080       93824992235648
r13          0x7fffffffde70      140737488346736
r14          0x0                0
r15          0x0                0
rip          0x555555551ed       0x555555551ed <main+132>
eflags      0x206              [ PF IF ]
cs          0x33                51
ss          0x2b                43
ds          0x0                0
es          0x0                0
fs          0x0                0
gs          0x0                0
(gdb)

```

Fig. 3 – GDB continued: pointer inspection (ptr, *ptr), arr[1]=2 via Indexed addressing, full step-through of all lines, and 'info registers' showing rax, rsp, rip, eflags and segment registers.

```

(gdb) disassemble main
Dump of assembler code for function main:
0x000055555555169 <+0>:    endbr64
0x00005555555516d <+4>:    push   %rbp
0x00005555555516e <+5>:    mov    %rsp,%rbp
0x000055555555171 <+8>:    sub    $0x30,%rsp
0x000055555555175 <+12>:   mov    %fs:0x28,%rax
0x00005555555517e <+21>:   mov    %rax,-0x8(%rbp)
0x000055555555182 <+25>:   xor    %eax,%eax
0x000055555555184 <+27>:   movl   $0xa,-0x2c(%rbp)
0x00005555555518b <+34>:   movl   $0x14,-0x28(%rbp)
0x000055555555192 <+41>:   movl   $0x1,-0x14(%rbp)
0x000055555555199 <+48>:   movl   $0x2,-0x10(%rbp)
0x0000555555551a0 <+55>:   movl   $0x3,-0xc(%rbp)
0x0000555555551a7 <+62>:   mov    -0x2c(%rbp),%eax
0x0000555555551aa <+65>:   add    $0x5,%eax
0x0000555555551ad <+68>:   mov    %eax,-0x24(%rbp)
0x0000555555551b0 <+71>:   mov    -0x2c(%rbp),%edx
0x0000555555551b3 <+74>:   mov    -0x28(%rbp),%eax
0x0000555555551b6 <+77>:   add    %edx,%eax
0x0000555555551b8 <+79>:   mov    %eax,-0x24(%rbp)
0x0000555555551bb <+82>:   mov    0x2e4f(%rip),%eax    # 0x55555558010 <x>
0x0000555555551c1 <+88>:   mov    %eax,-0x24(%rbp)
0x0000555555551c4 <+91>:   lea    -0x2c(%rbp),%rax
0x0000555555551c8 <+95>:   mov    %rax,-0x20(%rbp)
0x0000555555551cc <+99>:   mov    -0x20(%rbp),%rax
0x0000555555551d0 <+103>:  mov    (%rax),%eax
0x0000555555551d2 <+105>:  mov    %eax,-0x24(%rbp)
0x0000555555551d5 <+108>:  mov    -0x10(%rbp),%eax
0x0000555555551d8 <+111>:  mov    %eax,-0x24(%rbp)
0x0000555555551db <+114>:  mov    -0x24(%rbp),%eax
0x0000555555551de <+117>:  mov    %eax,%esi
0x0000555555551e0 <+119>:  lea    0xe1d(%rip),%rdi    # 0x55555556004
0x0000555555551e7 <+126>:  mov    $0x0,%eax
0x0000555555551ec <+131>:  callq  0x55555555070 <printf@plt>
=> 0x0000555555551f1 <+136>:  mov    $0x0,%eax
0x0000555555551f6 <+141>:  mov    -0x8(%rbp),%rcx
0x0000555555551fa <+145>:  xor    %fs:0x28,%rcx
0x000055555555203 <+154>:  je     0x5555555520a <main+161>
0x000055555555205 <+156>:  callq  0x55555555060 <__stack_chk_fail@plt>
0x00005555555520a <+161>:  leaveq
0x00005555555520b <+162>:  retq
End of assembler dump.

```

Fig. 4 – 'disassemble main' output: x86-64 assembly confirming Immediate (add \$0x5), Direct (mov via rbp offsets), Indirect (lea + mov via pointer), Indexed (mov with scaled index), and RIP-relative access for global variable x.

7. Observations

- **Immediate Addressing:** 'result = a + 5' compiled to *add \$0x5, %eax* — the constant 5 is part of the instruction itself.
- **Direct Addressing:** 'result = a + b' used MOV instructions with base-pointer offsets to fetch both variables from the stack.
- **Data Segment (Global):** 'result = x' used a RIP-relative MOV instruction to access the global variable from the data segment.
- **Indirect Addressing:** 'ptr = &a' stored the address 0x7ffffffdd54 in ptr. '*ptr' dereferenced it to yield 10.
- **Indexed Addressing:** 'result = arr[1]' accessed memory at arr's base address + 4 bytes (1 × sizeof(int)). GDB confirmed value = 2.
- **Registers:** rax held computed results, rsp pointed to the stack frame, rip held the next instruction address, and eflags reflected arithmetic flags.
- The single combined program allowed all five modes to be verified in one continuous GDB session.

8. Result

All five addressing modes — Immediate, Direct, Data Segment (Global), Indirect, and Indexed — were successfully demonstrated in a single C program and verified using GDB. The disassembled output confirmed how each C statement maps to a specific x86-64 assembly addressing mode at the machine level.